Finding min and max concurrently

Suppose you have an array of n distinct numbers. In the ICS 30-series, you learned how to find the min or max of such an array. Suppose you wanted to find *both* – the min and max.

One way to do this would be to find the min; this takes $n-1$ comparisons. You could then output and delete the min element and find the max of what remains, taking $n-2$ comparisons, for a total of $2n-3$ comparisons.

Question 1. Can you find a way to find both using *strictly fewer than* $2n-3$ *comparisons*? Note that we are measuring the actual number of comparisons, not the growth rate of your function.

If you are having trouble starting, you may assume n to be odd or even (your choice).

Question 2. Could anyone solve this problem in fewer comparisons than your solution uses?

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Counting Inverted Pairs

Associated reading: §11.1, Goodrich/Tamassia text.

Question 3. Suppose we have a `std::vector` with n distinct comparable elements (such as numbers). An *inverted pair* is a pair of indices i and j such that $i < j$ and $A[i] > A[j]$. That is, inversion occurs when the larger element of the pair appears earlier in the array. How many inverted pairs are in the following vector? What are they?

85	24	63	45	17	31	96	50

The following is an $\Theta(n^2)$ time way to count the inversions in an array. Note that we typically index arrays/vectors from $1 \dots n$ in CompSci 161.

```
count = 0
for i = 1...n do
  for j = i + 1...n do
    if A[i] > A[j] then
      count++
return count
```

The paradigm we will now cover is **Divide and Conquer** algorithms, whose associated problems can often be solved in polynomial time by brute force, but the technique can give us a more efficient solution.

Question 4. How many inverted pairs can a vector of size n have?

Question 5. Suppose instead the `std::vector` is arranged such that the first half is sorted, as is the second half. However, this does not mean the entire vector is sorted. For example, it might look like as follows; the halves are separated for effect.

Half 1:

41	48	51	66	68	84	87	89

Half 2:

14	23	37	43	52	62	96	98

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

We don't always start with two sorted lists, though. How can we make use of the previous part to count the inverted pairs in the entire vector?

Question 6. What is the running time of this approach?

This is a friendly reminder that tomorrow's discussion is required (in the sense that the material covered in it will be tested). Discussion will cover a better way to analyze the running time of Divide and Conquer algorithms

Reinforcement. Sometimes, it is good to trace an algorithm as you review, walking through how it works while also reviewing its design and correctness. Here is an array to use for this process. We encourage you to do so while the lecture is fresh in your mind.

48	41	51	66	84	89	87	68	37	23	96	98	52	14	62	43

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

QuickSort

Related reading: G/T §8.2

The key subroutine in QuickSort is `partition`.

48	41	51	66	84	89	87	68	37	23	96	98	52	14	62	43

```
QuickSort(A, start, end)
```

if start $<$ end **then**
  q = partition(A, start, end)
  QuickSort(A, start, $q-1$)
  QuickSort(A, $q+1$, end)

Average Case Analysis of QuickSort. Suppose:

- All permutations are equally likely
- All n values are distinct (for simplicity)
- Define $S_1, S_2, \ldots S_n$ as sorted order.

Question 7. What would have to happen for us to compare S_i and S_j?

Question 8. Let $P_{i,j}$ be probability we compare S_i and S_j. What is $P_{i,j}$?

Let $X_{i,j}$ be an indicator random variable for whether or not i, j get compared. Because that is a binary outcome, $X_{i,j} = 1 \cdot P_{i,j} + 0 \cdot (1 - P_{i,j}) = P_{i,j} = P_{i,j}$

The following is most of how we will show the expected number of comparisons made by randomized QuickSort is $\mathcal{O}(n \log n)$. *I will not ask you to write a proof like the following for credit, but I do expect that you should be able to understand this proof by the time of your first exam.*

$$\begin{aligned} E(\sum_{i=1}^{n}\sum_{j=i+1}^{n} X_{i,j}) &= \sum_{i=1}^{n}\sum_{j=i+1}^{n} E(X_{i,j}) \\ &= \sum_{i=1}^{n}\sum_{j=i+1}^{n} \frac{2}{j-i+1} \\ &= \sum_{i=1}^{n}\sum_{k=2}^{n-i+1} \frac{2}{k} \\ &< \sum_{i=1}^{n}\sum_{k=1}^{n} \frac{2}{k} \end{aligned}$$

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Reinforcement: Sort the following array by QuickSort. Use whichever mechanism you want for pivot selection. You might need more lines than provided.

87	31	30	22	20	85	86	15	38	60	57	72	41	52	50	67	69	3	65	42

Order Selection

Let's take a look at the *order selection* problem: the goal is to find the kth smallest element in an unsorted list. That is, the element that would be S_k when sorted. For example, if $k = 1$, we are looking for the smallest element in the list, while if $k = n$ we would be looking for the largest.

`Select(`S`, `k`)`
  If n is small, brute force and return.
  Pick a random $x \in S$ and put rest into:
    L, elements smaller than x
    G, elements greater than x
  if $k \leq |L|$ **then**

  else if $k == |L| + 1$ **then**

  else

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Deterministic Selection

Recall quickSelect from a few lectures ago. In this, we selected a pivot uniformly at random, performed a partition step, and if the pivot was not the desired kth smallest, we recursed appropriately. The expected case was $\mathcal{O}(n)$ time, but the worst case was $\mathcal{O}(n^2)$. Today, we will see a deterministic algorithm that will take $\mathcal{O}(n)$ time.

The only change to the algorithm is in pivot selection. Instead of selecting uniformly at random, we will divide the input into $g = \lceil n/5 \rceil$ groups of five elements each.

Question 9. How long does it take to find the median of any particular group of size five?

Question 10. How long does it take to find the median of those medians?

870	647	845	742	372	882	691	341	461	596
989	151	100	729	101	397	825	587	363	283
595	524	930	259	133	955	620	970	430	280
839	139	735	590	782	913	378	474	255	739
875	150	791	779	792					

Question 11. Suppose the resulting pivot *is not* the element we desire. What is the largest size the vector upon which we make a recursive call can be?

100	283	255	133	341				
101	363	378	259	461				
151	397	474	524	596	620	735	742	791
				691	955	782	845	792
				882	970	839	870	875

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Integer Multiplication

Reading: G/T §11.2

Given two n-bit integers X and Y, compute $X \times Y$. The algorithm you learned for this in grade school takes time $\mathcal{O}(n^2)$.

For our divide-and-conquer algorithm, we are going to divide X and Y each into their "higher order" and "lower order" bits first; X_H is the $n/2$ higher-order bits, and X_L is the lower-order bits.

Example If $X = 156 =$ `10011100` and $Y = 225 =$ `11100001`, then:

X_H	X_L	Y_H	Y_L
`1001`	`1100`	`1110`	`0001`

Note that $X = X_H \times 2^{n/2} + X_L$ and $Y = Y_H \times 2^{n/2} + Y_L$

Initial Algorithm Using algebra, we can see that

$$\begin{aligned} X \times Y &= (X_H \times 2^{n/2} + X_L) \times (Y_H \times 2^{n/2} + Y_L) \\ &= X_H \cdot Y_H \times 2^n + (X_H Y_L + X_L Y_H) \times 2^{n/2} + X_L Y_L \end{aligned}$$

Finish the Algorithm:

`Algorithm Mult(`X, Y`)`
  Create X_H, X_L, Y_H, Y_L
  A = Mult(X_H, Y_H)

Question 12. That's four recursive calls, each of size $n/2$, plus some addition, which takes an additional $\mathcal{O}(n)$ time. Why isn't this a good algorithm for computing $X \times Y$? Can we do better?

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Additional Practice

The two textbooks for the class have some wonderful questions to help you practice using the Divide and Conquer paradigm to design algorithms. Here are some suggested practice questions; note that we *will not* be posting a set of solutions for these. The process of working through the problem is far more important for you at this stage.

In the textbook of Goodrich and Tamassia, C-11.3, A-11.3, A-11.4, A-11.5

In the textbook of Erickson, Chapter 1, 9, 10(a), 11, 12, 14(a)(b), 17, 18, 22 (a)(b), 27, 28, 30, 31, 33, 36(a)

Master Theorem

Reading: Goodrich/Tamassia §11.1.1

It is common for a divide-and-conquer algorithm's running time to have a recurrence relation of the following form:

$T(n) = aT(n/b) + f(n)$, for some $a \geq 1$, $b > 1$, and $f(n)$ is asymptotically positive.

1. If there is a small constant $\varepsilon > 0$ such that $f(n)$ is $\mathcal{O}(n^{\log_b a - \varepsilon})$, then $T(n)$ is $\Theta(n^{\log_b a})$

2. If there is a constant $k \geq 0$, such that $f(n)$ is $\Theta(n^{\log_b a} \log^k n)$, then $T(n)$ is $\Theta(n^{\log_b a} \log^{k+1} n)$

3. If there is a small constant $\varepsilon > 0$ such that $f(n)$ is $\Omega(n^{\log_b a + \varepsilon})$, then $T(n)$ is[1] $\Theta(f(n))$.

Using the Master Theorem

Use the Master Theorem to solve the following:

1. $T(n) = 4T(n/2) + n$
2. $T(n) = 2T(n/2) + n \log n$
3. $T(n) = T(n/3) + n$
4. $T(n) = 9T(n/3) + n^{2.5}$

Using the Master Method

After we cover the Master Method, consider doing these as extra practice.

5. $T(n) = 2T(n/2) + 1$
6. $T(n) = 2T(n/2) + n$
7. $T(n) = 2T(n/2) + n^2$
8. $T(n) = 2T(n/4) + 1$
9. $T(n) = 2T(n/4) + \sqrt{n}$
10. $T(n) = 2T(n/4) + n$
11. $T(n) = 9T(n/3) + n$
12. $T(n) = T(2n/3) + 1$
13. $T(n) = 3T(n/4) + n \log n$
14. $T(n) = 2T(n/4) + n^2$
15. $T(n) = 2T(n/4) + n^4$
16. $T(n) = T(7n/10) + n$
17. $T(n) = 16T(n/4) + n^2$
18. $T(n) = 7T(n/3) + n^2$
19. $T(n) = 7T(n/2) + n^2$

[1] Technically, it must also be the case that $af(n/b) \leq \delta f(n)$ for some constant $\delta < 1$ and for all sufficiently large n. I will not give you any recurrence relations in CompSci 161 that fail to meet this condition.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.